

Skill 3 — Script Generator

Tricentis Competitive Intelligence Research Agent • Logic Specification

Trigger Phrases

- "Add [Competitor] to the pipeline"
- "Create scripts for [Competitor]"
- "Start tracking [Competitor]"
- "Make [Competitor] a tracked competitor"

Precondition Check

Before generating any files, verify both conditions:

Check	Condition	Action
1	Has Skill 2 already been run for this competitor in this conversation?	Yes — use URLs already discovered. No — run Skill 2 first, then return here.
2	Does scrapers/<competitor-slug>/ already exist?	Yes — tell the user and stop. Do not duplicate existing scrapers.

Step 1 — Confirm URLs

Present all discovered URLs organized by source type and require explicit user confirmation before generating any files. User must reply "confirmed" or adjust URLs first.

- Competitor Website URLs
- Social Media URLs — LinkedIn, X/Twitter, YouTube @handle + resolved channel ID (UCxxxxxxx)
- Analyst Coverage URLs
- Review Site URLs

Step 2 — Generate Four Scraper Files

Using the AccelQ scrapers as the reference template, create all five files:

File	Purpose
<code>scrapers/<slug>/competitor.py</code>	Website and core product URLs

<code>scrapers/<slug>/social_media.py</code>	LinkedIn, X/Twitter, YouTube RSS feed (resolved channel ID)
<code>scrapers/<slug>/analyst.py</code>	Forrester search URL, Gartner Peer Insights if available
<code>scrapers/<slug>/review.py</code>	PeerSpot reviews + vendor page, TrustRadius if available
<code>scrapers/<slug>/__init__.py</code>	Empty init file

Each file must include:

- Module docstring explaining what the scraper targets
- `COMPETITOR = "<slug>"` constant
- `URLS = [...]` with inline comments labeling each URL
- Standard `run(output_dir: Path) -> dict` function
- `if __name__ == "__main__":` block

social_media.py must also include:

- `from scrapers.base import scrape_multiple_with_rss`
- `RSS_URLS = ["https://www.youtube.com/feeds/videos.xml?channel_id=<resolved_ID>"]`

Step 3 — Update main.py

Add the new competitor slug to the `SUPPORTED_COMPETITORS` list in `main.py`:

```
SUPPORTED_COMPETITORS = ["opentext", "playwright", "uipath", "accelq", "keysight",
"<new-slug>"]
```

Step 4 — Require User Approval Before Pushing

Present all generated files to the user and require explicit approval before creating a branch, committing, or opening a PR. **Never push without confirmation.**

Step 5 — Push and Open PR

Once user approves:

- Create branch: `feat/add-<competitor>-scraper`
- Commit all 5 files (4 scrapers + `__init__.py`) and the `main.py` update
- Push branch to origin
- Open PR — Title: `feat: add [Competitor] to CI scraping pipeline`
- PR Body: competitor description, full URL list by source type, note generated from ad-hoc research on [date]

Step 6 — Confirmation

■ PR #[number] is open and ready for your review. Once merged, [Competitor] will be scraped automatically on the Monday/Wednesday/Friday schedule. The URL health check will also begin monitoring their URLs — no additional setup needed.

Key Constraints

- Never push to the repository without explicit user approval
- Always use AccelQ scrapers as the reference template — file structure must match exactly
- YouTube RSS requires the resolved channel ID (UCxxxxxxx), not just the @handle
- Never duplicate an existing competitor — always check scrapers/ first
- Never echo API keys, credentials, or secrets in any output